Payment Gateway | Setup & Demo

A practical walkthrough for the Java assessment implementation. Prepared 10 November 2026. Run commands from the repository root unless stated otherwise.

1. Prepare the environment

For the fully containerized route, see the Docker section on page 5; no host JDK is needed. For local development, install JDK 17, Docker with Docker Compose, and Python 3. Start Docker Desktop. The Gradle wrapper is included; Python smoke tests use only the standard library. The first Gradle build and Docker image pull need network access.

```
java -version
docker compose version
python3 --version
```

Confirm Java reports version 17. In IntelliJ, select JDK 17 for both Project SDK and the Gradle JVM. In a macOS terminal, if the JDK is registered:

```
export JAVA_HOME=$(/usr/libexec/java_home -v 17)
export PATH="$JAVA_HOME/bin:$PATH"
```

If discovery fails, set JAVA_HOME directly to your JDK 17 installation directory. On Windows, use gradlew.bat and the local Python launcher as appropriate.

2. Start the supplied bank simulator

```
docker compose up -d bank_simulator
docker compose ps
```

Expect bank_simulator to be running. Port 8080 serves the bank payment endpoint; 2525 is the Mountebank administration port. Keep the supplied imposters configuration unchanged.

3. Check and start the application

```
./gradlew test
./gradlew bootRun
```

Wait for Spring Boot to finish startup. Leave this terminal open to show correlated request logs. The gateway listens on port 8090 and defaults BANK_BASE_URL to http://localhost:8080. Automated Java tests do not require Docker.

Open http://localhost:8090/swagger-ui/index.html for the API documentation. In another terminal, confirm readiness:

```
curl http://localhost:8090/actuator/health/readiness
```

Expected: HTTP 200 with status UP. Readiness alone does not prove the bank is reachable; the payment smoke tests establish that integration.

Demonstrate the payment flow

4. Explain the components

The payment controller validates merchant input and calls the payment service. The service coordinates idempotency, maps the request to a bank DTO, calls BankClient, and stores an immutable masked response. BankClient translates transport and malformed-response errors. The repository uses an in-memory ConcurrentHashMap.

Packages separate payment, bank, and shared observability concerns. Validation runs at the controller boundary. Both authorized and declined payments are stored; invalid requests are rejected before calling the bank.

5. Submit a payment and retrieve it

```
curl -i http://localhost:8090/payment \
  -H 'Content-Type: application/json' \
  -H 'Idempotency-Key: demo-payment-001' \
  -d '{"card_number":"2222405343240001",
       "expiry_month":12,"expiry_year":2099,
       "currency":"GBP","amount":1050,"cvv":"012"}'
```

Expected: HTTP 201, status Authorized, a gateway UUID, cardNumberLastFour 0001, currency GBP and amount 1050. The response includes expiryMonth and expiryYear, but no full card number or CVV. GBP 1050 minor units represents GBP 10.50.

```
curl -i http://localhost:8090/payment/REPLACE_WITH_RETURNED_ID
```

Replace the placeholder with the actual returned UUID. Expect HTTP 200 and the same response data. GET reads stored data and does not call the bank.

6. Show idempotency and alternative outcomes

Repeat the POST unchanged with the same key: expect the same ID and body. Change amount to 200 while retaining that key: expect 409 IDEMPOTENCY_CONFLICT. Use a fresh key for each new payment demonstration.

With a fresh key, card ending 2 returns HTTP 201 with status Declined. Ending 0 causes HTTP 503 BANK_UNAVAILABLE. The simulator authorizes odd final digits and declines 2, 4, 6 and 8. A decline is a completed payment outcome, not a technical failure.

Only uppercase GBP, USD and EUR are accepted. Try JPY or a one-digit CVV: expect HTTP 400 with status Rejected. An unknown payment UUID returns 404. The independent scripts on the next page perform these checks automatically.

Run individual smoke checks

The gateway and Docker simulator must be running. Each script creates its own required state, so scripts can run in any order. All commands below run from the repository root. PASS means exit code 0; FAIL means a nonzero exit code.

`python3 scripts/smoke/authorized.py`
201 authorization, masked response and matching GET

`python3 scripts/smoke/currencies.py`
GBP, USD and EUR accepted and preserved by GET

`python3 scripts/smoke/invalid_currency.py`
JPY, malformed, null and missing currency rejected

`python3 scripts/smoke/invalid_cvv.py`
Invalid CVV rejected with 400

`python3 scripts/smoke/declined.py`
201 Declined and matching GET

`python3 scripts/smoke/replay.py`
Same key and payload return the original payment

`python3 scripts/smoke/conflict.py`
Changed payload with the same key returns 409

`python3 scripts/smoke/bank_unavailable.py`
Simulator 503 maps to BANK_UNAVAILABLE

`python3 scripts/smoke/missing_payment.py`
Unknown UUID returns 404

`python3 scripts/smoke/health.py`
Health, liveness, readiness and info endpoints

`python3 scripts/smoke/metrics.py`
Custom/HTTP metrics and Prometheus export

`python3 scripts/smoke/correlation.py`
Correlation header preserved on success and rejection

`python3 scripts/smoke/management_exposure.py`
Environment endpoint returns 404

Run everything or override the address

```
python3 scripts/smoke-test.py
python3 scripts/smoke/authorized.py --base-url http://localhost:8091
```

The base URL option works for every script and the combined runner. Use these synthetic payment requests only against the simulator-backed gateway. Runs create in-memory payment/key entries and increment metrics. Metrics tests seed their own activity so they work on a freshly started gateway.

Observability, limits & troubleshooting

7. Demonstrate operational visibility

```
curl http://localhost:8090/actuator/metrics/gateway.bank.requests
curl http://localhost:8090/actuator/metrics/gateway.payment.attempts
curl http://localhost:8090/actuator/metrics/gateway.idempotency.requests
curl http://localhost:8090/actuator/prometheus
```

Show X-Correlation-ID in a payment response and find it in the gateway terminal. Logs record method, route, HTTP status and duration without card data. A caller-supplied canonical UUID is preserved and forwarded to the bank. IDs and idempotency keys are not metric labels.

Bank/payment timers measure actual attempts; replays do not add bank attempts. Idempotency decisions and retained entries, handled payment errors, standard HTTP/JVM metrics and Prometheus export are available. Some meters appear only after the corresponding activity. See docs/observability.md for details.

8. Explain current boundaries

Storage and idempotency state are local to one process and disappear on restart. Keys have no expiry or eviction. This is not a distributed exactly-once guarantee. Bank timeouts can leave the payment outcome unknown; automatic retries are deliberately absent.

Smoke checks exercise the real HTTP gateway and supplied Docker simulator. They do not prove concurrency safety, inspect bank call counts, or exercise timeouts/malformed upstream JSON. Automated Java tests cover those cases with substituted bank transport. Monitoring endpoints are unauthenticated for local assessment use; no Prometheus server or tracing backend is included.

9. Diagnose common problems

Connection refused: wait for startup, confirm port 8090, and check docker compose ps. If payments return 503 unexpectedly, inspect BANK_BASE_URL and simulator logs with docker compose logs bank_simulator.

Port already in use: use the already-running gateway or stop the conflicting process before starting another. If you deliberately change the gateway port, pass --base-url to the smoke scripts.

Gradle/JDK failure: verify JAVA_HOME and IntelliJ Gradle JVM both select JDK 17. For a first build, allow Gradle to download dependencies.

10. Stop and reset

```
# Stop bootRun with Ctrl+C in its terminal.
docker compose stop bank_simulator
```

Restarting the gateway resets stored payments, keys and metrics. The scripts generate fresh keys and future expiry dates, so repeated runs do not need manual cleanup.

Reference: Checkout.com assessment requirements. Project references: README.md, docs/observability.md and scripts/smoke/README.md.

Run the complete stack in Docker

Build, test and start

Start Docker Desktop, then run these commands from the project root. The build uses JDK 17 to run the Java tests and produce the executable JAR. The final image contains a Java 17 JRE and runs as non-root UID 10001. No host JDK or Gradle installation is required.

```
docker compose up -d --build --wait
docker compose ps
python3 scripts/smoke-test.py
```

The first build downloads images, OS packages and Gradle dependencies. Python 3 is needed on the host only for the smoke scripts. The --wait option waits for gateway health; payment smoke tests then verify bank connectivity.

Networking and health

Compose publishes the gateway at http://localhost:8090, bound to loopback. The gateway calls http://bank_simulator:8080 through the Compose network. Inside a container, localhost would refer to the gateway itself, so Compose explicitly sets BANK_BASE_URL.

The image checks /actuator/health/readiness every 10 seconds, with a 40-second startup grace period. The simulator starts before the gateway, but has no readiness probe. Gateway health does not call the bank. Use the smoke tests to check the full payment flow.

Keep a local gateway running alongside Docker

```
GATEWAY_PORT=8091 docker compose up -d --build --wait
python3 scripts/smoke-test.py --base-url http://localhost:8091
```

Use http://localhost:8091/swagger-ui/index.html for this container. Retain the same GATEWAY_PORT value on subsequent Compose up commands. The container still listens on 8090 internally; only the published host port changes.

Logs, rebuilding and shutdown

```
docker compose logs -f gateway
# After source changes:
docker compose up -d --build --wait
# Stop the stack:
docker compose stop
# Or remove containers and the Compose network:
docker compose down
```

Use the port override for rebuilds too if you selected 8091. Ctrl+C exits log-following without stopping the containers. Stopping or recreating the gateway loses its in-memory payments and idempotency keys. The scripts create fresh test state on each run.

This provides local packaging and a reproducible build/test route. It does not deploy publicly or add persistent storage, distributed idempotency, authentication or a monitoring backend.